بسمه تعالی

📘 فایل آموزشی C# پیشرفته

تفکیک‌شده برای دو سطح: Mid-Level و Senior

تهیه و تنظیم : علیرضا بهمنش راد - https://www.linkedin.com/in/behmaneshrad

---

منبع:

https://learn.microsoft.com/en-us/training/browse/?expanded=dotnet&terms=c%23

https://roadmap.sh/ai/course/

نقشه مسیر آموزش

1. اجرای برنامه‌های C# و نقش CLR
2. Class، Method و Object (واقعی نه تمرینی)
3. Reference vs Value (با دید باگ‌یابی)
4. Collections مهم بک‌اند
5. Exception در دنیای واقعی
6. Resource Management
7. Interface و Dependency
8. تمرین پایانی شبه‌پروژه

بخش 1 – اجرای برنامه C# (واقعیت Backend)

مفاهیم کلیدی

- C# مستقیماً اجرا نمی‌شود
- کد → CPU → CLR → IL
- Runtime با Compile-Time فرق دارد

✅ تمرین 1 – درک Runtime

```
csharp
class Program
{
    static void Main()
    {
        int x = 10;
        int y = x;
        y = 20;

        Console.WriteLine(x);
    }
}
```

سؤال:

- خروجی چیست؟
- چرا؟

📌 هدف تمرین: آماده‌سازی ذهن برای Value vs Reference

بخش 2 – Class و Object در دنیای واقعی

اشتباه جونیورها

- ساخت کلاس بدون مسئولیت مشخص

مثال واقعی: User در Backend

```
csharp
class User
{
    public string Name;
    public string Email;
}
```

❌ مشکل:

- Encapsulation ندارد
- اعتبارسنجی ندارد

نسخه بهتر (قدم اول به Mid-Level)

```
csharp
class User
{
    public string Name { get; }
    public string Email { get; }

    public User(string name, string email)
    {
        if(string.IsNullOrWhiteSpace(email))
            throw new Exception("Invalid Email");

        Name = name;
        Email = email;
    }
}
```

✅ این یعنی فکر بک‌اند

✅ تمرین 2

- کلاس `Product` بساز
- قیمت منفی نباشد
- نام خالی نباشد

بخش 3 – Reference vs Value (ریشه ۷۰٪ باگ‌ها)

مثال مشکل‌دار رایج

```
csharp
void UpdatePrice(Product product)
{
    product.Price = 100;
}
```

✅ تغییر می‌کند چون Reference است

اما این چه؟

```
csharp
void Increase(int price)
{
    price += 10;
}
```

❌ تغییری بیرون رخ نمی‌دهد

✅ تمرین 3

1. یک struct به نام `Point`
2. یک class به نام `PointClass`
3. هر دو را به متد بده
4. تفاوت رفتار را مشاهده کن

بخش 4 – Collections مهم Backend

لیست ضروری

- List
- Dictionary
- HashSet

مثال واقعی

```
csharp
Dictionary<int, User> users = new Dictionary<int, User>();
users[1] = new User("Ali", "ali@mail.com");
```

📌 چرا Dictionary؟

- Lookup سریع
- کلید منطقی

✅ تمرین 4

- لیست سفارش‌ها
- جستجو بر اساس OrderId
- چه Collection مناسب است؟ چرا؟

بخش 5 – Exception Handling واقعی

اشتباه مرگبار جونیورها

```
csharp
try
{
    // code
}
catch
{
}
```

❌ یعنی دفن کردن خطا

الگوی درست

```
csharp
try
{
    SaveOrder(order);
}
catch(InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
}
```

✅ تمرین 5

- متدی بنویس که اگر قیمت صفر بود Exception بدهد
- فقط Exception مرتبط را catch کن

بخش 6 – Resource Management

مفهوم حیاتی

- فایل
- دیتابیس
- شبکه

مثال با using

```
csharp
using(var file = new StreamWriter("log.txt"))
{
    file.WriteLine("Log");
}
```

✅ تضمین آزاد شدن منابع

✅ تمرین 6

- فایل بساز
- متن بنویس
- عمداً Exception بده
- ببین فایل بسته می‌شود یا نه

بخش 7 – Interface (شروع تفکر معماری)

بدون Interface (بد)

```
csharp
class OrderService
{
    SqlRepository repo = new SqlRepository();
}
```

❌ وابستگی شدید

با Interface (گام میدلول)

```
csharp
interface IOrderRepository
{
    void Save();
}

csharp
class OrderService
{
    private IOrderRepository _repo;

    public OrderService(IOrderRepository repo)
    {
        _repo = repo;
    }
}
```

✅ تمرین 7

- Interface برای Payment بساز
- دو پیاده‌سازی (Fake و Real)

بخش 8 – تمرین نهایی (شبه پروژه)

سناریو

سیستم ساده ثبت سفارش

نیازمندی‌ها

- Order
- Product
- OrderService
- Validation
- Exception
- Interface

✅ تمرین نهایی

1. مدل‌ها Immutable باشند
2. قیمت منفی نباشد
3. Repository از Interface استفاده کند
4. مدیریت Resource رعایت شود

🟢 بخش اول – آموزش مفاهیم Mid-Level Backend #C

این بخش دانشی است که **هر توسعه‌دهنده Backend با سطح Mid-Level باید بدون تردید بلد باشد**

---

1 CLR (Common Language Runtime)

تعریف

CLR محیط اجرای برنامه‌های دات‌نت است که کد نوشته‌شده را مدیریت و اجرا می‌کند.

وظایف CLR

- اجرای IL Code
- مدیریت حافظه (Memory Management)
- Garbage Collection
- Thread Management
- Exception Handling
- Security

📌 نکته مهم آموزشی

کدی که با #C می‌نویسید مستقیماً اجرا نمی‌شود، بلکه ابتدا به IL و سپس به کد ماشین تبدیل می‌شود.

---

2 IL Code (Intermediate Language)

تعریف

زبانی میانی که خروجی کامپایلر #C است و قبل از اجرا توسط CLR به کد ماشین تبدیل می‌شود.

چرا IL مهم است؟

- مستقل از سیستم‌عامل
- اجازه اجرای یک برنامه روی چند پلتفرم مختلف
- JIT بر اساس شرایط Runtime بهینه‌سازی می‌کند

---

3 JIT Compilation

تعریف

فرآیند تبدیل IL Code به Native Machine Code در زمان اجرا.

پیامدها

- عملکرد نزدیک به Native
- امکان بهینه‌سازی بر اساس CPU و محیط واقعی

---

4 Value Type و Reference Type

Value Type

- داده مستقیماً نگهداری می‌شود
- هنگام Assignment کپی می‌شود
- مثال: `int`، `struct`

Reference Type

- فقط Reference کپی می‌شود
- داده روی Heap قرار دارد
- مثال: `class`، `array`

📌 اثر مستقیم روی:

- Performance
- Behavior متدها
- Thread Safety

---

5 struct vs class

struct مناسب است وقتی:

- حجم داده کوچک است
- Immutable است
- نیاز به Inheritance ندارد

❌ struct نامناسب:

- برای Objectهای بزرگ
- در سناریوهای پرتکرار Method Call

---

6 Garbage Collection (GC)

GC چیست؟

سیستمی در CLR که حافظه اشیای بدون مرجع را آزاد می‌کند.

GC چه کاری نمی‌کند؟

- فایل را نمی‌بندد
- Connection دیتابیس را آزاد نمی‌کند
- Socket را نمی‌بندد

📌 GC فقط **Managed Memory** را مدیریت می‌کند.

---

7 Managed vs Unmanaged Resources

Managed Resource

منابعی که زیر نظر CLR هستند (Objectها)

Unmanaged Resource

منابع سیستم‌عامل:

- File Handle
- Database Connection
- Network Socket

---

8 Dispose و Finalizer

Dispose

- فراخوانی صریح
- کنترل قطعی
- روش استاندارد آزادسازی منابع

Finalizer

- غیرقطعی
- زمان اجرای نامشخص
- فقط در شرایط خاص

✅ همیشه **Dispose را ترجیح بده**

---

9 Exception Handling

Exception چیست؟

مکانیزمی برای گزارش خطای غیرمنتظره در Runtime.

اصول مهم

- Exception برای Flow عادی نیست
- ایجاد Exception پرهزینه است
- `catch(Exception)` خطرناک است

---

10 Interface

Interface چیست؟

قراردادی که مشخص می‌کند یک کلاس چه متدهایی باید داشته باشد.

کاربردها

- کاهش Coupling
- طراحی معماری تمیز
- جداسازی لایه‌ها
- Dependency Injection

---

11. Dependency Injection (DI)

تعریف

تزریق وابستگی‌ها از بیرون به‌جای ساختن داخلی.

مزایا

- تست‌پذیری
- انعطاف‌پذیری
- پیاده‌سازی SOLID (DIP)

---

12. Abstraction

تعریف

نمایش رفتار بدون نمایش جزئیات پیاده‌سازی.

هدف

- کاهش پیچیدگی
- کنترل تغییرات
- طراحی پایدار

🔵 بخش دوم – آموزش مفاهیم #Senior Backend C

این بخش دانشی است که **نداشتنش = Senior نبودن**

---

1 CLR Memory Model

چیستی

مدلی که مشخص می‌کند:

- Threadها چگونه حافظه را می‌بینند
- ترتیب خواندن و نوشتن چگونه است

اهمیت

بدون درک Memory Model، برنامه Multi-thread امن نیست.

---

2 Thread Visibility

تعریف

اینکه تغییرات یک Thread چه زمانی توسط Thread دیگر دیده شود.

❌ بدون تضمین Visibility:

- باگ‌های Production
- رفتار غیرقابل پیش‌بینی

---

3 Reordering

تعریف

تغییر ترتیب اجرای دستورات توسط:

- CPU
- JIT

- CLR

نتیجه

- Race Condition
- Bugهای تصادفی

---

4 volatile

volatile چه می‌کند؟

- تضمین Visibility
- جلوگیری از Cache شدن مقدار

❌ چه کاری نمی‌کند؟

- Atomicity عملیات پیچیده

---

5 Immutability

تعریف

شیئی که بعد از ساخته‌شدن تغییر نمی‌کند.

مزایا

- Thread Safety طبیعی
- Debug آسان
- کاهش Lock

---

6 Readonly vs Immutable

Readonly

- Reference تغییر نمی‌کند
- State داخلی می‌تواند تغییر کند

Immutable

- Reference و State هر دو ثابت

---

7 async / await

واقعیت async

- async یعنی Non-blocking
- async ≠ Thread جدید

کاربرد مناسب

- IO-Bound Operations
- Backend Scalability

---

8 Deadlock

تعریف

حالتی که Threadها منتظر یکدیگر می‌مانند.

علت رایج در async

- SynchronizationContext
- `.Result` یا `.Wait()`

---

9 Task vs ValueTask

Task

- ساده
- امن برای استفاده عمومی

ValueTask

- کاهش Allocation
- مخصوص سناریوهای High-Performance
- استفاده نادرست = Bug

---

10 API Contract

Contract چیست؟

تعهد بین Backend و Client.

اهمیت

- Breaking Change = Production Issue
- Versioning حیاتی

---

11 DTO vs Domain Model

DTO

- فقط داده
- بدون Business Logic

Domain Model

- شامل منطق کسب‌وکار
- نباید مستقیم بیرون منتشر شود

---

12 Allocation Thinking

تعریف

تفکر درباره:

- طول عمر Object
- تعداد Allocation
- فشار روی GC

---

13 Premature Optimization

تعریف

بهینه‌سازی قبل از:

- Measurement
- Bottleneck واقعی

❌ نتیجه:

- کد پیچیده
- نگه‌داری سخت